How your Boat Rental site works

A beginner-friendly guide to the current files in **C:/xampp/htdocs/IT37**. Prepared 10 October 2026. This explains the code as it exists now, including the buttons, saved records, calculations, and current limitations.

The main idea

The browser loads the page, then JavaScript handles most of your actions. A sidebar click shows a different section. A Save button checks the inputs and writes records into the browser. The site then rebuilds the visible tables and totals from those records.

XAMPP serves the files through localhost. The two pages have a .php extension, but their current contents are HTML; they do not query MySQL. There is no database connection, PHP session, or server-side record storage in this version.

File	What it does
index.php	Login screen: username, password, show-password checkbox, and login button.
main.php	All management screens, navigation links, dashboard cards, and shared dialogs.
main.js	Login checks, click handlers, forms, validation, saved records, calculations, filters, and rendering.
styles.css	Colors, typography, spacing, tables, boat cards, responsive layouts, and print rules.
assets/boat-logo.png	Speedboat logo used on the login page and sidebar. CSS displays it in white on dark backgrounds.

Reading map

2: Login and navigation. **3:** Events and forms. **4:** Storage and record relationships. **5:** Resources and images. **6:** Clients and staff. **7:** Booking validation. **8:** Payments. **9:** Cancellation. **10:** Reports. **11:** Dashboard and availability. **12:** JavaScript reference. **13:** Complete example and troubleshooting.

Source: the local index.php, main.php, main.js, and styles.css inspected for this guide. Examples are explanatory and are not inserted into the site. The site was not changed or run for this document.

Login and moving between screens

There are two kinds of navigation: opening another page and showing another section of the existing page.

1. Login opens main.php

The login form has the ID login-form. main.js attaches a submit listener to it. When you press Login, it reads the username and password. The username is trimmed to remove spaces at the beginning and end; the password is compared as entered.

```
if (username === 'admin123' && password === 'admin123') {
    window.location.href = 'main.php?role=admin';
} else if (username === 'staff123' && password === 'staff123') {
    window.location.href = 'main.php?role=staff';
}
```

An incorrect combination shows an error message and stays on the login page. The Show password checkbox changes the password input between type="password" and type="text". It changes visibility only; it does not change the password.

2. The address controls the displayed role

The query string is the part after the question mark. URLSearchParams reads role=staff. Only that exact value sets isStaff to true; the code otherwise uses admin. The role changes the greeting and controls. Staff cannot add or edit resources through the normal interface, and New booking starts with the boat-card screen.

This is a demonstration login. The URL determines the role, and there is no server verification or session. The staff records you add are rental personnel; they do not create new login credentials.

3. Sidebar links switch sections

```
<a href="#resources" class="nav-item">Resources</a>
```

```
window.addEventListener('hashchange', showScreen);
```

The hash is the part after #. showScreen() finds the section whose ID matches the hash, displays it, and sets hidden=true on the others. It also changes the page title, highlights the navigation item, closes the mobile menu, and scrolls to the top. An empty or unknown hash falls back to Dashboard.

Action	Destination / effect	Saves records?
Resources / Clients / Staff	Matching section in main.php	No
Dashboard summary card	Resources, Availability, Bookings, or Payments	No
My Account / profile	The account section	No
Logout / Back to login	index.php; stored records remain	No

What an Add button actually does

The Add buttons reuse one dialog and one Save handler. The button tells the code which kind of record you want.

The button supplies a label the code can read

```
<button type="button" data-form="clients">
    + Add client
</button>

button.addEventListener('click', function () {
    openRecordForm(button.dataset.form);
});
```

The custom HTML attribute data-form becomes button.dataset.form in JavaScript. Clicking this button passes the string "clients" into openRecordForm(). The word must match a key in formLayouts, which holds the title and field definitions for each form.

openRecordForm(kind, editId, resourceId)

kind identifies resources, clients, staff, bookings, or payments. **editId** is normally empty, but contains an existing resource or staff ID when editing. **resourceId** preselects a boat when Book is pressed from a row or card.

The function reads stored records, checks prerequisites, clears the previous inputs, and creates labels, input fields, or dropdowns. It uses the existing record to prefill editable fields. It stores the kind and editId in the dialog dataset so Save knows what to do. dialog.showModal() displays the pop-up.

Save is a form submission event

```
event.preventDefault();
savePortalRecord();

// Inside the save function:
const values = Object.fromEntries(new FormData(previewForm));
const kind = dialog.dataset.kind;
```

preventDefault() stops a normal browser form submission and reload. FormData reads the named fields, and Object.fromEntries turns them into an object. For example, values.name contains the text from the input named "name". Required fields and input types provide browser checks before the submit event runs.

savePortalRecord() disables Save while it is working, reads the latest records, and performs the checks for that record type. It adds or replaces an object, calls writeRecords(), closes and resets the form, and shows a short message. The finally block enables Save again even after an error.

Closing is different from cancelling a booking

The form Cancel button or X calls dialog.close(). Unsaved inputs are not written to storage. The separate Cancel booking action changes an already saved booking; that flow is explained on page 9.

Where the data goes

All five record lists are saved as one JSON value in localStorage, under the key boatRental.sharedRecords.v1.

```
{
    resources: [],
    bookings: [],
    clients: [],
    staff: [],
    payments: []
}
```

Reading and writing

```
const saved = localStorage.getItem(storageKey);
const records = JSON.parse(saved);

localStorage.setItem(storageKey, JSON.stringify(records));
renderPortal();
```

localStorage stores text. JSON.stringify converts the object into text, and JSON.parse turns that text back into objects and arrays. readRecords() returns empty lists when there is no saved value. For older saved data, it adds missing clients, staff, and payments lists in memory without deleting existing resources or bookings. Invalid list structures cause an error.

writeRecords() saves the entire records object under the same key, then calls renderPortal(). Calling renderPortal() by itself only reads and displays data; it does not save it. If storage is blocked or full, saving shows an error instead of reporting success.

List	Fields stored
resources	id, name, type, capacity, rate, status, image
clients	id, name, email, phone, address
staff	id, name, role, phone, email, status
bookings	id; clientId/client; resourceId/resourceName; staffId/staffName; start, end, guests, total, status; bookedBy, bookedRole, createdAt; cancellation fields
payments	id, bookingId, amount, method, reference, createdAt

Why IDs matter

newId() calls crypto.randomUUID() to create a unique identifier. A booking keeps the IDs of the client, boat, and assigned staff. A payment keeps its bookingId. These IDs connect records even when two people have the same name. Bookings also keep name snapshots for display, so editing a staff name does not automatically rewrite old bookings.

Browser storage is tied to the browser profile and origin: protocol, host, and port. localhost and 127.0.0.1 use different storage. Switching admin/staff in the same profile and origin shares records; another browser or computer does not. Refreshing normally keeps data, but clearing site data removes it. There is no automatic server backup in this version.

Resources, editing, and pictures

A resource is a boat in the fleet. Its saved status describes whether it can be booked; time conflicts are checked separately.

Adding a resource

The administrator presses Add resource. The form collects name, type, capacity, rate per hour, status, and an optional image. Staff are blocked both when opening this form and when attempting the resource save branch.

Field	Current rule
Name / type	Must contain non-space text.
Capacity	Whole number from 1 to 1,000.
Hourly rate	Greater than zero and at most 999,999.99; saved to two decimal places.
Fleet status	Dropdown offers Available, Maintenance, and Inactive.
Image	Optional image file, no larger than 2 MiB (2 x 1,024 x 1,024 bytes).

What happens to the image

The file input gives JavaScript a File object. readImage(file) creates a FileReader and returns a Promise. The reader loads the image as a data URL: a string containing the image type and encoded image data. await pauses this save operation until reading finishes.

```
const uploadedImage = imageFile ? await readImage(imageFile) : '';

// A saved resource contains the image string:
image: imageFile ? uploadedImage : (existing ? existing.image || '' : '')
```

The image is saved inside the resource record, not uploaded into an XAMPP images folder. Tables use it as the thumbnail source, and staff boat cards use it as the large photo. A resource without an image gets a placeholder on the card. Large images use a significant amount of browser storage.

Editing keeps the same ID

Edit opens the form with the existing details. A file input cannot be prefilled with an image string, so it stays empty. If you do not choose another image, the existing one is retained. Save replaces the resource object at its array position while keeping its ID.

Before saving an edit, the code looks for non-cancelled bookings that have not ended. It prevents changing the boat away from Available while those bookings exist. It also prevents reducing capacity below the guest count of any such booking. Existing booking totals are stored separately and are not recalculated when the hourly rate changes.

Available versus Booked

The resource may remain saved as Available but display Booked while a rental is currently in progress. That display is calculated from the dates. A boat open for bookings can still have future reservations; the booking form checks the exact requested times before saving.

Clients and staff members

The booking form selects saved people by ID. It does not create a client or staff member just because a name is typed.

Adding a client

Add client opens fields for full name, email, phone, and address. These are required. The browser checks the email input format, and the save code trims each string and rejects blank values. A new object with a unique ID is added to records.clients, then writeRecords() persists the updated lists.

The Clients table shows the name and address, email, phone, and number of bookings linked to that clientId. The booking count includes cancelled bookings because they remain part of the client history. The current interface has Add client, but no client Edit or Delete action.

Why the booking dropdown uses saved clients

openRecordForm() fills the client dropdown with saved records, sorted by name. Each option displays the name, email, and phone, but its value is the client ID. On Save, the code looks up that ID again in the latest records. An empty or missing client cannot be booked.

```
const client = records.clients.find(function (item) {
    return item.id === values.clientId;
});

if (!client) {
    throw new Error('Choose a client registered on the Clients page.');
}
```

Adding and editing staff

Add staff collects full name, role, phone, email, and status. The status is Active or Inactive. The code requires the details, stores an ID, and adds the member to records.staff. Edit prefills the same form and replaces the selected member while preserving the ID. The current interface permits both portal roles to add and edit staff.

A role such as Captain is descriptive text. It does not give someone administrator access. The actual portal still uses the fixed admin123 and staff123 login combinations. Staff status also does not represent a work schedule or shift calendar.

How staff availability is checked

The Assigned staff dropdown includes Active members with no overlapping, non-cancelled booking. Changing departure or return runs updateStaffOptions(), which rebuilds the options and retains the selected member only if still available. Save repeats the availability check against the latest saved records.

Before dates are chosen, active staff can appear because no valid time conflict has been specified yet. The final save still requires valid dates. Adding a staff member supplies a rental assignee; it does not sign you in as that individual.

Creating a booking, step by step

This is the main connection between clients, resources, staff, dates, and charges.

Opening the correct form

For staff, New booking without a selected resource first changes the hash to #choose-boat. The card list can be filtered by name, type, capacity, and fleet status. Book this boat calls openRecordForm("bookings", "", resource.id). For admin, New booking opens the form directly. A Book action on a resource row also supplies its ID.

The code checks that at least one client, an available resource, and an active staff member exist. If any prerequisite is missing, it directs you to the relevant screen. The card selection preselects the boat; you still fill in client, departure, return, guests, and assigned staff.

Checks made at Save

The selected client must exist. The boat must have saved status Available. Both dates must be valid, and return must be later than departure. Guests must be a positive whole number within boat capacity. Assigned staff must exist, be Active, and be free during that period. The boat must also be free. The code rejects periods whose rounded hours exceed 365 days.

```
new Date(start) < new Date(otherEnd)
    && new Date(end) > new Date(otherStart)
```

This is overlaps(): the requested rental starts before another rental finishes and ends after that rental starts. Both conditions mean the periods overlap. A booking ending exactly at 12:00 does not overlap one starting at 12:00. Cancelled bookings are ignored by this check.

How the price is calculated

```
const hours = Math.ceil((end - start) / 3600000);
const total = Math.round(resource.rate * 100) * hours / 100;
```

Date subtraction gives milliseconds. One hour is 3,600,000 milliseconds. Math.ceil rounds the duration upward to a whole hour. The rate is converted to centavos for the multiplication and converted back to pesos afterward. A PHP 500 hourly rate for 2 hours 15 minutes costs 3 x PHP 500 = PHP 1,500.

What is stored and where it goes next

The saved booking includes linked IDs, copied display names, start/end, guest count, total, Confirmed status, timestamp, and the login that created it. bookedBy is admin123 or staff123; staffId is the assigned member. They are different fields. After saving, staff are taken to Bookings. Dashboard counts, availability, pending balances, and any generated report are redrawn.

Payments and balances

Record payment stores a receipt against a booking. The payment status is calculated from recorded money, rather than entered manually.

The Record payment button

openRecordForm("payments") first checks for at least one non-cancelled booking with a remaining balance. If there are none, it shows a message. Otherwise, the booking dropdown lists eligible bookings with their reference, client, and current balance.

The form collects booking, amount, payment method, and an optional reference. Available methods are Cash, GCash, Bank transfer, and Card. Save rejects missing or cancelled bookings, non-positive amounts, amounts above the current balance, and unrecognized methods. The payment date is the current save time; there is no backdated-payment field.

```
records.payments.push({
    id: newId(),
    bookingId: booking.id,
    amount: cents / 100,
    method: values.method,
    reference: values.reference.trim(),
    createdAt: new Date().toISOString()
});
```

The three calculation functions

receivedPayment(records, bookingId) adds payments linked to the booking and returns centavos. **paymentBalance(records, booking)** subtracts that sum from the booking total and never returns less than zero. **bookingPaymentStatus(records, booking)** returns Cancelled first for a cancelled booking, otherwise Paid, Partial, or Unpaid.

Booking total	Received	Balance	Status
PHP 1,500	PHP 0	PHP 1,500	Unpaid
PHP 1,500	PHP 500	PHP 1,000	Partial
PHP 1,500	PHP 1,500	PHP 0	Paid

What the Payments page displays

renderPayments() groups information by booking. A booking with several receipts is one row showing the combined amount, remaining balance, and distinct payment methods. References are available for searching. A non-cancelled booking with no receipt can still appear as Unpaid. Cancelled bookings with recorded payments remain visible; cancelled bookings with no payments are omitted.

A partial payment changes the balance but the Pending Payments count stays the same because money is still owed. A full payment removes that booking from the pending count. The current application records payments only; choosing GCash or Card does not contact a payment provider or charge money.

Cancelling a booking

Cancellation is a saved change to an existing booking, with a record of who was selected as the person cancelling.

Opening the cancellation dialog

The Cancel booking row button calls cancelBooking(id). This function rereads storage and looks up that booking. If it is missing or already cancelled, it shows a message. Otherwise, it fills the Cancelled by dropdown and displays the booking reference, client, and boat in the cancellation dialog.

The dropdown contains active staff members. When the portal role is admin, it also contains Administrator (admin123). A staff portal cannot select the administrator option. If a staff portal has no active member available to select, the operation directs the user to the Staff page.

Confirm cancellation performs the actual write

Opening the dialog alone makes no record change. The cancel-form submit listener rereads the latest records and verifies that the booking still exists and is not cancelled. It also checks that the selected staff member is still Active, or that Administrator was selected from the admin portal.

```
booking.status = 'Cancelled';
booking.cancelledBy = admin ? 'Administrator (admin123)' : member.name;
booking.cancelledByStaffId = admin ? '' : member.id;
booking.cancelledByRole = admin ? 'admin' : 'staff';
booking.cancelledAt = new Date().toISOString();
writeRecords(records);
```

The selected name is stored as a snapshot, together with role, staff ID where relevant, and cancellation time. The table displays the name and formatted time. Because login is still the simple prototype, this is a recorded selection, not proof that the selected employee personally signed in.

What changes afterward

Area	Effect
Booking history	The record stays visible with Cancelled status; it is not deleted.
Boat / staff availability	The cancelled booking no longer creates a time conflict.
Dashboard	It is excluded from upcoming bookings and pending-payment counts.
Reports	It remains in matching rental history, with the cancellation details. Booked value and outstanding totals exclude it.
Existing payments	They remain saved. Cancellation does not remove receipts or issue refunds.

Keep booking or the X closes this dialog without cancelling. The current site has no restore-booking button and no refund workflow. Cancellation does not automatically create a separate financial transaction.

Generating and printing reports

Reports calculate a view of existing records. They do not create new bookings, payments, or permanent report files inside the app.

Choosing the report

Report offers Rental overview and Payments. Payments reveals a status dropdown with All payments, Fully paid, Partially paid, Unpaid, and Cancelled bookings. The removed Still to be paid option is no longer part of the current interface.

Dates default to the first day of the current month through today. Resource and Client default to All. updateReportFilters() fills these dropdowns from saved records and also includes identities referenced by older bookings. Older bookings without clientId use a separate name-based option labeled older bookings.

What Generate report does

The submit handler checks that both dates are present and From is not after To. It stores the chosen dates, IDs, labels, report type, and payment filter in the report heading dataset. These are temporary page values, not saved localStorage records. renderPortal() then calls renderReport().

renderReport() filters bookings by the departure date, selected resource ID, selected client identity, and, for Payments reports, payment status. Both date endpoints are included. A rental that starts before the date range is excluded even if it continues into that range. The rows are ordered by departure time.

Summary	How it is calculated
Bookings	Count of the matching booking rows, including cancelled rows if the selected filters include them.
Booked value	Sum of matching non-cancelled booking totals.
Total paid	Sum of recorded payments linked to the matching bookings, including receipts kept for cancelled bookings.
Total to be paid	Sum of each matching non-cancelled booking's remaining balance. Cancelled rows contribute zero.

The date meaning is important

These are **payments for rentals departing in the selected period**, not a cash-receipts report filtered by payment.createdAt. For example, a payment recorded on 5 October for a rental departing on 10 October appears when the selected rental dates include 10 October. This matches the Departure from / Departure to labels.

Rental overview shows booking status and cancellation details. Payments shows financial status and payment methods. Changing report controls hides the old result and disables Print until Generate is pressed again. This prevents printing results under newly edited, unapplied controls.

Printing

Print report calls window.print(). The browser opens its print dialog; Save as PDF can save a copy. Print CSS hides navigation, controls, and footer, and formats the report table. The printed heading includes the applied dates and filters. No report is automatically emailed or uploaded.

Dashboard, availability, and refresh

The display is rebuilt from the saved records. A displayed number is often calculated rather than stored as its own field.

Dashboard cards

Card	Current calculation
Total resources	Number of objects in records.resources, including inactive or maintenance boats.
Available	Boats saved as Available with no non-cancelled booking currently in progress.
Bookings	Non-cancelled bookings whose end is later than now. This includes rentals already underway.
Pending payments	Non-cancelled bookings whose remaining balance is greater than zero. This is a booking count, not a peso total.

The Upcoming Bookings table sorts those not-yet-ended bookings by start time and shows the first five. A booking does not automatically change its saved Confirmed status to Completed when its end passes. The date-based dashboard count changes even though the stored status does not.

Availability screen

Previous day, Next day, Today, and the date input change which date is displayed. They call updateSchedule(), which updates the heading and refreshes the view. They do not create reservations or save the selected day as a record.

renderSchedule() divides a day into three blocks: Morning 00:00-12:00, Afternoon 12:00-18:00, and Evening 18:00-24:00. Any overlapping non-cancelled booking marks that block Booked. A short rental can therefore mark the whole block Booked; the booking form still checks exact times. Maintenance and Inactive take priority over calculated availability.

Search and filters

Search listeners use the input event, so the view updates while typing. Dropdowns use change. matchesFilters() compares lower-case search text and the selected status. Filtering changes visible rows only. Boat-card filters also check boat type and minimum guest capacity; Clear filters restores their default values.

When renderPortal() runs

It runs during page initialization, after saving a record or cancelling a booking, after relevant search/filter/date changes, and when the browser window regains focus. A storage event also refreshes another open tab when the same origin's records change there. The tab performing a save is refreshed directly by writeRecords().

renderPortal() calls functions that create table cells, badges, buttons, boat cards, payment rows, filter options, and report rows. It does not continuously run every second. Time-based availability can remain unchanged on a page left open until another refresh-triggering action happens.

JavaScript terms and function map

Use this page while reading main.js. Each term connects to a task the site actually performs.

Term	Meaning in this project
const / let	const prevents reassignment of the variable itself; let allows it. A const object can still have its properties changed.
Object / array	An object groups named fields, such as a client. An array is a list, such as records.clients.
Function	A named block of work. openRecordForm() opens a form; money() formats an amount.
Event listener	Runs code when something happens: click, submit, input, change, hashchange, storage, or focus.
DOM	The browser's in-memory page structure. getElementById and querySelector find its elements.
dataset	Custom data-* attributes exposed to JavaScript. Used for form type, edit ID, and applied report selections.
find / findIndex	Find one matching record or its position in an array. Used to look up and edit by ID.
filter / some	filter creates a matching list; some asks if at least one match exists, such as a time conflict.
map / reduce	map transforms items; reduce combines them into one result, such as total received payments.
push / forEach	push appends a record; forEach performs work on each item, such as creating table rows.
async / await / Promise	Handles work that finishes later. Image reading returns a Promise; await waits before saving.
try / catch / finally	try attempts the work, catch handles errors, and finally restores Save-button state.
use strict	Enables stricter JavaScript behavior, such as errors for accidental undeclared variables. It is not an authentication feature.

The functions to learn first

showScreen(): section navigation. **openRecordForm()**: form construction. **savePortalRecord()**: validation and record updates. **readRecords() / writeRecords()**: persistence. **renderPortal()**: visible refresh. **overlaps()**: booking conflicts. **renderReport()**: report filtering and totals.

addCell(), addStatus(), and addAction() build reusable table pieces. money() and dateTime() format data for display. showMessage() displays a toast for about four seconds. These helpers keep the larger functions from repeating the same work.

The code uses textContent for entered names and labels, so they are inserted as text rather than interpreted as HTML. This does not turn the browser-only role system into server-side authorization.

One complete example

This example follows a record through the site. The names and amounts below are illustrations only; no sample data was added.

Step	What happens in the code
1. Add a boat	An admin saves Harbor One, capacity 5, rate PHP 500/hour, status Available. records.resources receives a new ID.
2. Add people	Add client Alex and active staff member Sam. Each gets a separate ID in the corresponding list.
3. Make a booking	Choose Harbor One, Alex, and Sam for 10 October, 10:00-12:15, with 3 guests. The code validates IDs, capacity, dates, and conflicts.
4. Calculate the charge	2 hours 15 minutes rounds up to 3 hours. The saved total is PHP 1,500. Pending Payments increases by one.
5. Record PHP 500	A payment object links to the booking ID. Received is PHP 500, remaining balance PHP 1,000, and status Partial.
6. Generate a report	Choose Payments, rental dates including 10 October, and optionally Harbor One or Alex. The report includes the matching booking and its totals.
7. Finish payment	Record another PHP 1,000. Received becomes PHP 1,500, balance becomes zero, status becomes Paid, and the pending count decreases.

If the booking were cancelled instead

The booking would remain in history with the selected canceller and timestamp. It would stop blocking the boat and staff. Its outstanding amount would be excluded from report totals, while any receipts would remain recorded. The app does not issue a refund.

If information appears to be missing

First check that you are using the same browser profile and localhost address. Check whether a search box, status selection, report dates, or report filters are hiding records. Reports include departure dates, so the default current-month range may exclude older rentals. Do not clear site data while investigating missing browser records.

No records in localStorage produces empty lists; malformed or blocked storage produces an error message. Updating the HTML, CSS, or JavaScript does not itself move records into MySQL. The current code does not automatically back up or synchronize localStorage to another device.

Current limits to remember

The fixed login is a UI prototype. My Account displays Saved but does not persist its fields. Staff records do not create accounts. There is no database, automatic backup, refund flow, or automatic Completed status. Payments record money received rather than process a charge. These are descriptions of the current implementation, not features added by this guide.

For explanation in class: "The interface sends events to JavaScript. JavaScript validates inputs, updates objects, saves them as JSON in localStorage, and renders the current records back into the page."